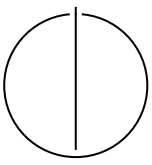
SCHOOL OF COMPUTATION,
INFORMATION AND TECHNOLOGY —
INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Master's Thesis in Informatics

Improving Register Allocation in a Fast Compiler

Thomas Saller



SCHOOL OF COMPUTATION, INFORMATION AND TECHNOLOGY — INFORMATICS

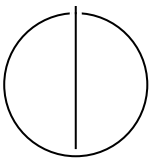
TECHNISCHE UNIVERSITÄT MÜNCHEN

Master's Thesis in Informatics

Improving Register Allocation in a Fast Compiler

Verbesserung der Registerallokation in einem schnellen Compiler

Author:	Thomas Saller
Examiner:	Prof. Dr. Thomas Neumann
Supervisor:	Dr. Alexis Engelke
Submission Date:	08.04.2026



I confirm that this master's thesis is my own work and I have documented all sources and material used.

Munich, 08.04.2026 Thomas Saller

Acknowledgments

Abstract

Fast compilation is crucial in contexts where code generation occurs at runtime, such as query compilers in modern database systems or runtimes for modern programming languages. However, the quality of initially generated code is often markedly suboptimal, prompting many systems to adopt multi-tiered compilation strategies.

Recently, TPDE has shown that compilation speed of LLVM unoptimized back-end can be improved substantially while keeping a similar runtime. However, because TPDE further reduces compilation time at the unoptimized level, it also widens the compile-time gap to the next optimization level, `-O1`, to two orders of magnitude.

In this work, we present a novel register allocation approach built on top of TPDE that enables improved code generation while preserving its core single-pass code-generation model. By combining two analysis passes with a repair mechanism applied during code generation, our approach optimizes spill placement and resolves instruction constraints.

On SPECint 2017 we achieve a 24% runtime improvement over the LLVM -O0 back-end, while maintaining a 4x faster compile time on optimized LLVM IR. Although, this also results in a 5x compile time slowdown compared to the original TPDE. Our results demonstrate the potential for a faster, lightly optimizing compiler back end; however, several design and implementation issues limit the performance of our approach.

Contents

1 Introduction

Just-in-time (JIT) compilation is a widely used technique for improving performance for applications, where the combined speed of compilation and execution is critical. Such applications include dynamic runtimes such as the Java Virtual Machine [PVC01] as well as the acceleration of database queries [Neu11].

Since compile time directly contributes to end-to-end latency, fast machine code generation is especially important for JIT compilation. Recently, TPDE [SKE26] has shown that the compilation speed of the LLVM [LA04] unoptimized back-end can be improved substantially, while keeping a similar runtime performance.

Since JIT systems inherently involve a trade-off between compilation and runtime, many systems employ multi-stage approaches, with the first stage serving as the baseline and subsequent stages being progressively optimized [SKE26]. However, when using LLVM, the difference in compilation time between TPDE and LLVM's optimized backend (`-O1`) is two orders of magnitude, leaving a significant gap for programs that could benefit from better code generation, but for which the substantial increase in compilation time cannot be justified.

To address this gap, we propose a novel register allocation approach built on top of TPDE that enables improved code generation while maintaining relatively fast compilation times. Our approach consists of two analysis passes: a liveness analysis, which additionally computes next-use information, and a spill analysis pass that determines which values to spill and optimizes spill locations to reduce spill overhead in loops. During code generation, we introduce a repair mechanism for handling instruction constraints, enabling more efficient code generation for constrained operations such as integer division on `x86-64`.

On SPECint 2017, we achieve a 24% runtime improvement over the LLVM-O0 backend, while the compilation time for optimized LLVM-IR remains three times faster. However, this also results in a 5x increase in compilation time compared to TPDE. Nevertheless, our work achieves a new Pareto optimum for the compilation of optimized IR, as it is significantly faster than LLVM-O1 in terms of compilation time, while simultaneously offering a runtime improvement over both TPDE and LLVM-O0.

The key contributions of our work include:

- A lifetime analysis that generates next-use information in the same pass
- A spill analysis that identifies values to spill without relying on knowledge of the instruction semantics, or requiring modification to the IR.
- A simplified repairing mechanism to handle instruction constraints during code generation, without rolling back code generation.

The remainder of this work is structured as follows: In Chapter 2 we provide background on register allocation and TPDE, as well as the constraints arising from the framework's design. In Chapter 3 we present our approach. In Chapter 4 we discuss related work in register allocation, and how it relates to our approach. Next we evaluate our approach compared to TPDE, LLVM-O0 and LLVM-O1 in Chapter 5. Finally, we summarize our results and discuss future work in Chapter 6.

2 Background

In this chapter, we will introduce the basic concepts of register allocation and the particular importance of the SSA form. We then briefly describe the TPDE backend framework and the resulting challenges building register allocation on top of it.

2.1 Register Allocation on non-SSA programs

Register allocation is the process of assigning values from a program to a limited number of k physical registers. This process is commonly modeled as finding the graph coloring of an interference graph (IG)corresponding to the program.

In an interference graph, nodes represent program values, while edges indicate interference between values, meaning that the connected values cannot be assigned to the same register. This formulation naturally corresponds to graph coloring, where adjacent nodes must not share the same color. Consequently, a valid coloring using at most k colors yields a feasible register allocation for the corresponding program on a perfectly regular architecture with at least k registers.

In practice, the direct application of such an approach is not feasible, since GC is NP-complete with respect to the number of values [GJ90], and any given IG can be used to construct a program with the same IG [Cha+81]. Furthermore, deciding how many registers a program requires or whether k registers are sufficient is also NP-complete [Hac07].

Practical RA must rely on heuristics at every step to find allocations in a reasonable amount of time. We will illustrate this using the seminal work by Chaitin [Cha+81] on the subject.

Note that we are assuming a theoretical regular architecture for now. We will address architectural constraints and the associated complications later.

2.1.1 Chaitin Style Graph Coloring Register Allocation

First, the interference graph itself must be constructed. This involves scanning the program and adding edges between values that are live simultaneously [Cha+81].

Next, a coalescing heuristic is applied to merge nodes stemming from copy instructions into a single node. Note that this step may increase the number of colors required

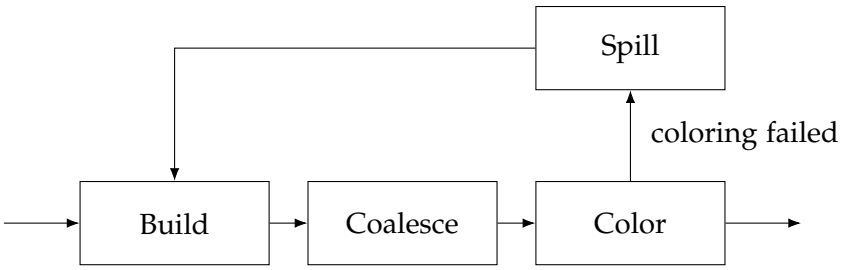


Figure 2.1: Simplified overview of the steps in a graph coloring allocator [Hac07]

for the new graph, since the merged nodes may have a higher degree than the original ones [Cha+81].

Finally, the main coloring step begins. First, all nodes with fewer than k neighbors are removed from the graph and placed into a stack. If the graph is now empty, we move on to the next step. Otherwise, a remaining node is selected for spilling and the IG is rebuilt for the new program after spill code is inserted. Spilling removes edges from the graph as the spilled value's lifetime is reduced. This process is repeated until the graph is empty [Cha+81].

First the Interference Graph itself has to be constructed. This is done by scanning the program and adding edges between values that are live at the same time.

Next, a coalescing heuristic is applied such that the nodes stemming from copy instructions are merged into a single node. Notice that this step might increase the number of colors required for the new graph since, the merged nodes may have a higher degree than the original nodes [Cha+81].

Finally, the main coloring step begins. First all nodes with fewer than k neighbors are removed from the graph and placed into a stack. If the graph is now empty, we move on to the next step. Otherwise, we select a remaining node for spilling and rebuild the IG for the new program after inserting spills. This removes edges from the graph, as the lifetime of the spilled value is reduced. This process is repeated until the graph is empty [Cha+81].

Now that all nodes are on the coloring stack, we can reinsert them into the graph and assign them a color. This is always possible since each node on the stack has fewer than k neighbors. This produces a valid coloring of the final program.

While this approach yields good results, building the inference graph is computationally expensive, and the iterative nature of the algorithm makes it impractical for use in a JIT compiler [CD06].

Although there have been multiple improvements to the original algorithm, the basic iterative structure remains the same, prompting the development of different approaches to the problem.

```
%a   = ...
%b   = ...
%c   = ...
%d   = add i32 %b, %c
%e   = add i32 %d, %a
%a_1 = i32 %e (redefinition of a)
```

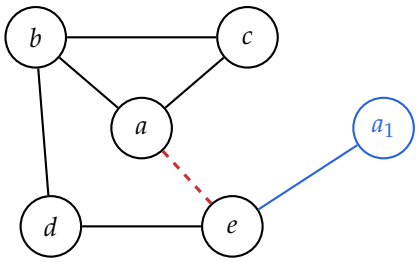


Figure 2.2: Example snippet of LLVM-IR. Assume that `%a_1` is a redefinition of `%a` in the original program. This would add the red edge in the interference graph, creating a cycle of length 4 with no other edges connecting the cycle nodes. Therefore, the graph with the red edge is not chordal. The SSA version adds a new node `%a_1` that removes the cycle, therefore the graph with the blue elements remains chordal.

2.1.2 Other Approaches

One alternate approach is to formulate Register Allocation as an Integer Linear Programming problem. While this can produce optimal results it is also NP-complete and the worst case runtimes of several minutes makes them unattractive for JIT compilers[FW02].

The other common approach is to use a linear scan allocator [PEK97]. This approach avoids the expensive IG data structure in favor of using a total ordering of all program instructions. This ordering is used to determine the single live range of each value, for example starting at instruction i and ending at instruction j ($[i, j]$). The live ranges are then assigned to registers in the order of their interval start.

Once the algorithm runs out of registers, it spills a value, splitting its live range into two parts. The spilled value is typically chosen by the longest remaining live range length.

This approach is simple and non-iterative and therefore attractive for use in a JIT compiler. However, due to the simplified liveness information, the resulting allocation quality is comparatively poor. Especially around control flow such as loops and branches with differing register pressure. Although improvements have been made to the original algorithm, the resulting allocation quality is considered worse than that of graph coloring [WM05].

2.2 Register Allocation on SSA programs

Limitations of previously discussed approaches primarily stemmed from the NP-

Figure 2.3: Simplified overview of the steps in an SSA register allocator [Hac07].

completeness of the underlying allocation problem. For SSA programs, the interference graph can no longer be arbitrary and must be chordal [HGG06].

Chordal graphs are graphs where every cycle of length four or more has a chord, an edge connecting two vertices in the cycle that is not itself part of the cycle (see Fig. 2.2).

Chordal graphs have a number of useful properties, including the property that they can be colored in polynomial time. Additionally, the required number of colors is equal to the size of the largest subset of vertices that are all adjacent in the graph. This subset is determined by the program points with maximal register pressure [Hac07].

SSA register allocators can therefore first use a spilling phase to reduce the register pressure to at most k at each program point, and then color the resulting chordal graph (this is possible without explicitly constructing the graph [Hac07]).

Note that these results still assume a regular architecture. Resolving PHI nodes with cycles and loops, as well as constrained instructions, may require additional registers. Splitting live ranges around constrained instructions can avoid some of these problems, but it introduces additional work in a later coalescing phase [Hac07].

As our work focuses on register allocation speed rather than perfect allocation quality, and we lack the necessary information to employ these techniques, we will not consider these issues further.

2.3 TPDE

TPDE [SKE26] is a compiler backend framework focusing on short compilation times. It can be used with most SSA intermediate representations without a translation step.

This work, however, will focus on LLVM-IR. This also entails that TPDE cannot modify the IR in any way.

In TPDE, functions consist of a sequence of basic blocks, which in turn consist of a sequence of instructions. Instructions take a number of operands and produce a number of results, both of which are values. Values can have multiple parts, where each part has a register bank and size associated. Each part can be stored in its designated stack slot or in a register at any program point.

Lifetime management occurs at the value level and is calculated using approximated liveness with a start and end basic block as well as its number of uses. After its last use, the value is considered dead, and its stack slot can be reused.

```
void emit_udiv64(IRInstr inst) {
 ValuePartRef lhs_ref = val_ref(inst->getOperand(0)); // Handle for operand 0
 ValuePartRef rhs_ref = val_ref(inst->getOperand(1)); // Handle for operand 1
 ScratchReg rdx_scratch{Reg::DX}; // Scratch for RDX, unspillable
 lhs_ref.into_specific(Reg::AX); // Move or load lhs into RAX
 ValuePartRef res_ref = result_ref_will_overwrite(inst, 0, std::move(lhs_ref));

 ASM(XORrr, rdx_scratch.cur_reg(), rdx_scratch.cur_reg());
 AsmReg rhs_reg = val_as_reg(rhs_ref); // Force into arbitrary GP register
 ASM(DIV64rr, rhs_reg); // Encode instruction, clobbers RDX, RAX
 res_ref.set_modified();
} // Release handles and scratch
```

Listing 2.1: Example instruction compiler for an 64-bit unsigned division instruction for x86-64. The framework will move values into registers and create a copy of the first operand if required. We use a ScratchReg to clear the `RDX` register.

TPDE combines instruction selection, register allocation and instruction encoding into a single code generation step. This is done by user provided instruction compilers for each IR instruction, which encode the instruction's semantics and compile them directly into machine instructions.

2.3.1 Instruction Compilers

Instruction compilers are responsible for getting handles to the actual values, which prevent them from being spilled during the compilation function. Then, they load the operands into registers or use them as memory operands and encode the instruction. Instruction compilers can also allocate scratch registers, which cannot be spilled and are used in situations where an instruction requires additional registers or clobbers registers other than their operands and results (see Listing 2.1).

Instruction compilers may allocate more or fewer registers than is suggested by the number of their operands and results. Since TPDE is unaware of the true semantics of the instruction, register allocation becomes more difficult, as it must occur before code generation.

2.3.2 Register Allocation in TPDE

Whenever an instruction compiler requests a register, the framework will check whether the part is in a register already. Otherwise, it will look for a free register from the correct bank. If no register is available, it will spill a register according to heuristic

taking into account the cost of reloads as well as the remaining lifetime of the spilled value. For requests of a specific register, the value contained in that register will be spilled.

In addition to this basic mechanism, values that are used across basic blocks in the innermost loop can be assigned a fixed register. These values will remain in the same register for their entire lifetime and will never be spilled. This is intended for loops to prevent spilling of loop induction values or other values used in the innermost loop.

Otherwise, all non-fixed registers are spilled at each basic block boundary, unless the next block is the immediate successor with only one predecessor. This avoids having to store the register state across basic blocks.

2.3.3 TPDE specific challenges for Register Allocation

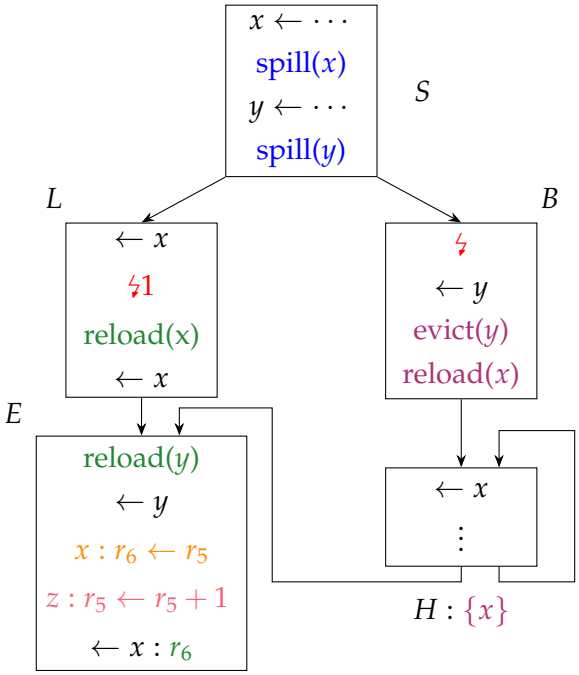
As previously established, instruction compilers may require more or fewer registers than is suggested by their operands and results. This makes register allocation perfect register allocation impossible, as the code generation step is final and intelligent register choices have to be calculated beforehand.

Fortunately, most instructions behave regularly in practice. To this end, we will make the following regularity assumptions, which we assume to hold for most instructions and programs:

- Most values have one part.
- Most instructions require exactly as many registers as their operands and results suggest.
- Most instructions do not need specific registers or clobber non-result registers.
- Most loops are executed often so that spills and reloads outside a loop are always preferred.
- Most loops are reducible.

Our work will utilize these assumptions while ensuring the approach remains correct, but potentially suboptimal in the presence of irregularity.

3 Approach



1. Lifetime analysis with calculation of next-use distances

2.1. Spilling pass to reduce register pressure at critical points (↯)

2.2. During spilling calculate optimal loop header state

3. Code generation with repairing at constrained instructions and insertion of reloads.

This chapter describes how we extend the TPDE framework to support better register allocation choices, maintaining a single-pass code generation.

First we perform a lifetime analysis that calculates the distance to the next use of a value at each point. Then a spilling pass marks registers to be spilled at points with high register pressure (↯), spilling the furthest next-use (x at B, and y at L). These values are then spilled after their definition. Additionally, we calculate a minimal register set for loop headers, that contains as many loop values as possible, while spilling non-loop values (y at the end of B).

During code generation we insert reloads as necessary to encode instructions. When encountering constrained instructions such as `add` on `x86-64`, we attempt to repair the register state by moving values to different registers. This prevents the values from being lost due to instruction constraints. At branches, we insert connecting code to ensure that the register state matches the target block, and the calculated loop header

state, if we branch to a loop header.

3.1 Liveness with next-use distance

As described in Section 2.3, the existing liveness analysis in TPDE operates at the basic block level and does not account for lifetime holes.

This is insufficient for an effective register allocation as it creates too many interferences between variables, that are not actually present in the original program.

To address the issue, we introduce a precise liveness analysis, that also calculates the next-use distance at each point in the program.

To accomplish this, we adapt the approach by Boissinot, Brandner, Darte, et al. [Boi+11].

We define the next-use as the number of instructions between *start of the current block* and the next use of the variable. Next-uses are therefore always strictly increasing within a block. The next-use distance can easily be calculated using the index of the current instruction and the next use.

3.1.1 Data structures

We chose to store this information as a sorted (by construction) list of next-uses for each variable and block. Definitions are marked by having their highest bit set. Therefore, a value is defined in a block if its first next-use entry has the highest bit set (and is not dead). Dead values have a next-use of `u32::max` or no entry if the value is neither live-in nor defined in the block. A value is live-out if its last entry is not `u32::max`.

For example, the following array describes a value that is defined in the second instruction of the block and used as an operand in the third instruction, after which it is dead.

0	1	2
`(1«31) | 1`	2	`u32::max`

Additionally, we apply a loop exit penalty to live-out distances if the successor is part of the same loop. This penalty is simply a large number added to the distance and accounts for the frequency with which loops are executed, according to our regularity assumptions (see Section 2.3). This encourages the later spilling pass to avoid spilling loop variables [BH09].

Overall, this allows us to find the next use of a value in logarithmic time relative to the number of uses in a basic block. As well as checking live-in, live-out and whether it was defined in the block in constant time. Additionally, checking whether a value

is used in a loop takes constant time, as we assume that all other uses are below the chosen loop exit penalty.

3.1.2 Partial liveness

First we calculate the partial liveness for each block by traversing the control flow graph (CFG) in post-order, skipping loop edges. We then process the instructions for each block in forward order while tracking the current instruction index and noting the definitions and uses of each variable.

To find the live-out distances, we iterate over the direct successors, while skipping loop edges and taking the minimum next-use from the successors. As we traverse the CFG in post order, the non-loop successor blocks are already processed.

Note that values used only in a ϕ definition are not considered live-in, but we will consider them live-out at their predecessor blocks with a use at the zeroth instruction of the successor.

3.1.3 Liveness propagation

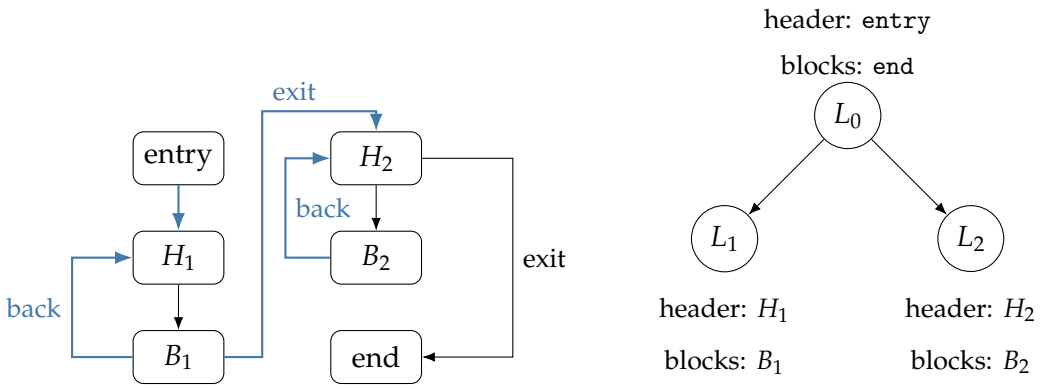
To propagate the liveness information through the program, we calculate a loop forest as described by [Boi+11], implemented using the existing TPDE loop analysis.

Liveness is propagated, by marking live-in values from a loop header as live throughout all child loops (the children of the loop header in the loop forest). This corresponds to adding a single next-use entry for each value in the children of the loop header. The next-use entry is larger than the block size indicating that the value is both live-in and live-out, but is neither used nor defined within the block.

Whenever a value is propagated to a different loop (including L_0), we add the loop exit penalty to the next-use entry.

For large functions, this can lead to many live-through values, which can be problematic as our original data structure may result in up to one allocation per entry created. Instead, we track the live-through values separately in a vector, which is later sorted by the index of the value and its next-use entry. This approach still allows for quick access (in logarithmic time relative to the number of live-through values in the block) while reducing the number of allocations to one per block.

Please note, that this calculation is not applicable to irreducible loops with multiple loop headers. As these loops are rare, we do not handle this case, as insufficient liveness information will not affect the correctness of the final generated program. For further details on how to adapt this liveness analysis for irreducible control flow, see [Boi+11].



(a) Control-flow graph (b) Loop nesting forest

Figure 3.1: A control-flow graph and its corresponding loop nesting forest [Boi+11]. The outermost loop L_0 is only the root of the tree, and does not get treated as a loop, this corresponds to the existing loop analysis in TPDE [SKE26]. The edges marked blue are loop edges, as they connect to a loop header, which are skipped on the partial liveness pass. On edges marked `exit` the loop exit penalty is added if a value is live after the loop.

3.2 Spilling

The primary purpose of the spilling pass is to ensure that the register pressure is less than the number of registers k. Typically, a spilling pass inserts spills and loads into the IR. For SSA form, this will also require inserting ϕ nodes as the reloaded value has to be distinct from the original value, to remain in SSA form.

Instead, we only store the results of the spilling pass, and ensure that blocks are correctly connected, with each value in one unique location during code generation.

3.2.1 Representing Spills and Reloads

Since we can't modify the existing IR, inserting the calculated spills and reloads is not possible. In particular, we would need to check at every instruction whether a reload or spill needs to be generated.

While this would be possible, it requires a lot of bookkeeping. Instead, we approximate the spill and reload locations. We avoid storing reloads, except for loop headers and will insert a reload if necessary during code generation.

For spills, we only store whether a value should be spilled according to the analysis. Then during code generation we insert the spill immediately after its definition. This simplifies analysis, as we don't need to track which vales where spilled in which

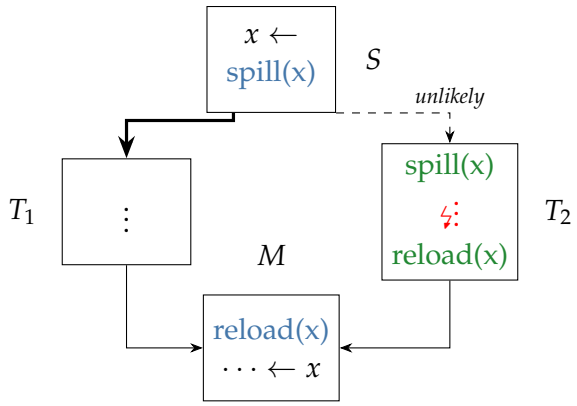


Figure 3.2: Example of a bad case for our simplified analysis. Green represents the ideal spill and reload locations. Blue is our approximation. T_2 is assumed to have high register pressure, therefore x has to be spilled. We assume that the branch to T_2 is unlikely, therefore we generate unnecessary spills and reloads.

branches, to avoid values being lost or unnecessarily spilled again. During code generation we only need to check a single bit in a bitset for the results, reducing the overhead of the spilling pass.

This approximation of spills does not result in the optimal solution in all cases, especially around blocks with very different execution frequencies (see Figure 3.2). This is particularly relevant for loops, which we assume to execute often.

To avoid cases like Figure 3.2 for loops, we store the ideal register set for the loop header separately. During code generation the appropriate spills and reloads are generated to match this register set. This ensures that as few spills and reloads as possible are generated within the loop, while still avoiding storing additional state for most basic blocks.

3.2.2 Calculating spills

We use a simplified `MIN` algorithm based on the work of [BH09].

For each block we simulate the working register set W by limiting the size of W to k registers. For values that are live but need to be removed from W, we mark them as spilled. Values with multiple parts, like 128-bit integers, always need to be fully in W or fully spilled.

We spill according to the next-use distance, once before the instruction for operands and once after the instruction to ensure space for the results.

At this stage, we do not consider any instruction irregularities, except for calls. Calls are handled by adding a implicit result for every volatile register. In addition, we mark values that are in W after a call and are used in the next 5 instructions, as these values are good candidates for non-volatile registers. All other irregularities are ignored (and are not known to TPDE at this stage).

Algorithm 2 Simplified `MIN` algorithm for a single basic block [BH09].

```
def LIMIT(W, insn, cap)
    sortByNextUse(W, insn)
    for v ∈ W[cap : −1] :
        if nextUse(v) ≠ ∞ :
            mark v as spilled
    W ← W[0 : cap]

def MINALGORITHM(block, W)
    for insn ∈ block.instr :
        W ← W ∪ insn.operands
        limit(W, insn, k)
        limit(W, insn.next, k − |insn.results|)
        W ← W ∪ {insn.results}
```

We start with W containing the function arguments. Then for every other block we initialize W with the intersection of all incoming values from predecessors. Since we don't insert reloads at this stage, the initialization is not critical, unless the current block is a loop header.

For loop headers, we initialize W to only contain variables used in the loop and restrict it to $k - p$ registers, where p is the approximate maximum register pressure in the loop. This pressure p is calculated during the liveness analysis and is simply the maximum number of live variables in any of the loop blocks. This ideally ensures that loop iterations can be executed without spilling. Since our register pressure p is only an approximation, we conservatively evict all values that are not used in the loop.

3.3 Register allocation during code generation

After the previous passes, we are now at the code generation stage of TPDE. Therefore, any generated instruction cannot be undone, as they are immediately encoded. This stage focuses on avoiding additional spills, other than those previously calculated, and handles connecting basic blocks together including ϕ nodes.

3.3.1 Register choice in a Basic Block

Register choice for unconstrained instructions

When a value requests a register, we choose in round-robin fashion from the currently unused registers. We prefer volatile registers, unless the value has been previously marked as a non-volatile candidate (see section 3.2.2). The chosen register is then noted

as the preferred register for this value. Future reloads of this value will try to use the same register if possible, in order to minimize connecting instructions [Col+11].

Should there be no free register, we need to spill a value. For this we choose the values with the largest next-use distance, which are typically already marked as spilled and therefore do not generate a new spill.

Register choice for constrained instructions

For constrained instruction, the associated compilation function (see Listing 2.1) will either use a Scratch register or request a specific register. Scratch registers are handled like normal values except that they cannot be spilled and no preferred register is stored as they are temporary. If a specific register is requested, and the register is currently in use, we move the occupying value into a free register if possible, thereby repairing the assignment instead of spilling. If no register is available, we spill the occupying value, since the previously used spill information is not correct around constrained instructions.

As requests are processed value by value it is possible, that we miss shuffling opportunities between operands. For example assume we have values x and y in registers rax and rdx respectively and all other registers are occupied. Now if we want to compile a 64-bit division, the compilation function will request rdx and zero it, which requires spilling y. Then y need to be in register rax which will spill x. And finally x needs an arbitrary register. This is not optimal as we could have used the final register of x as a temporary to swap x and y, only requiring one spill, which can also be an arbitrary register and therefore more likely to be marked spilled.

Although this scenario is possible, it is unlikely to occur as long as most instruction are unconstrained, values would also need to be in specific registers by chance and have a high register pressure at this point.

However, for calls such situations may be problematic, as all arguments have a specific location, and they occur frequently in programs. Therefore, we handle calls separately.

Register allocation for calls

For calls specifically all its operands need to be in a specific location either on the stack or in a specific register. To handle this better, we collect all operands and their target locations, then spill all volatile registers. Then in a second pass we first move all stack operands to their location, which frees up more registers. Then finally we use a parallel move algorithm [RSL08], which uses the free registers as temporaries in order to swap values between registers and resolve any dependency cycles between them.

3.3.2 Connecting Basic Blocks

At the end of each block we need to connect them with their successors. If the successor has only one predecessor, we can simply use the current register without any additional moves. For blocks with multiple predecessors, the register state of the first generated predecessor is used. If the current register state does not match the target state, we generate a parallel move to resolve the differences. We ensure this is possible by requiring one register per bank to be free in branch regions. This allows for the use of parallel moves to resolve any register conflicts. We try to minimize the amount of connecting moves necessary using the preferred register mechanism.

If a successor contains ϕ nodes, we try to allocate them to a register and move their incoming value to this register, this is done in the same parallel move as before. This can also handle all cycles or swaps as long as they are in registers. If the ϕ nodes are not in registers and conflict, we try to use the free register as an intermediate, otherwise we need to allocate an additional stack slot to resolve the conflict.

As we cannot undo any action taken during this connecting step we currently require all jumps to be split, although it would be possible to precalculate whether any moves, spills or reloads are necessary and then deciding whether a jump must be split.

Loop headers

As previously described, we consider loop headers as especially important and have therefore precalculated a register state during the spilling phase. This is used when connecting to a loop header. All non-loop values are spilled, and the missing values are reloaded into registers, to ensure the register state matches the precalculated one.

4 Related Work

As explained previously in Section 2.3, our work has access to less information than is typical, therefore all approaches discussed in this chapter would require significant adaptations to be applicable in TPDE.

To the best of our knowledge, there is no existing work, that is intended to be used without knowledge of the instruction selection and the instruction constraints.

4.1 Graph coloring Register Allocation

As previously described, graph coloring of interference graphs is equivalent to register allocation [Cha+81]. In general this is an NP hard problem [Cha+81]. For graphs that are not colorable with k colors, the graph coloring algorithms additionally need to iterate between coloring, spilling and coalescing. Where each spill step requires an expensive rebuild of the interference graph[CD06]. This makes traditional graph coloring register allocation too slow for just in time compilation [CD06]. There are multiple improvements proposed to the original algorithm aiming to improve the code quality, such as optimistic coalescing [PM04].

The cost of rebuilding the interference graph after every spilling step can be avoided by approximating the resulting interference graph using additional metadata [CD06]. This sacrifices a small amount of code quality for a significant speedup, which makes it more suitable for just in time compilation.

Compared to GC approaches our work adds only 2 preprocessing passes, including lifetime analysis, and does not require any iterative algorithm, which allows for a more predictable runtime.

We will consider such SSA-based algorithms such as ours separately as they have different compilation speed properties, even though they are fundamentally based on graph coloring.

4.2 Linear Scan Register Allocation

Linear Scan Allocators fundamentally of a single scan over the program, finding live ranges and then mapping these live ranges to registers (or memory) [PEK97].

Compared to Graph Coloring, Linear scan require only one scan over the program and no expensive interference graph construction. However, their code quality is generally worse than graph coloring [Hac07]. While some limitations of the original algorithm, such as the lack of lifetime holes for control flow, have been addressed[WF10]. Linear Scan Allocators fundamentally use a non-optimal approach with increasingly elaborate heuristics or post-processing passes to improve code quality[WF10].

Similar to improvements by Wimmer and Mössenböck[WF10], our approach precomputes a set of spilled variables, allowing for better spill locations compared to spilling once the algorithm runs out of registers.

Similarly, we emphasize loops placing spills and reloads outside of loops as much as possible. As opposed to the later modifications to Linear Scan, we cannot change allocated code afterwards, which limits the effectiveness of the proposed improvements. In particular, moving spills would not be possible for our work. Due to this limitation, our preprocessing steps are responsible for finding good spill locations. Since our code generation step remains as a single pass, our approach could be considered a modified variant of linear scan with additional preprocessing stages.

4.3 Trace Register Allocation

For JIT Compilers in particular, trace allocation can be used, which assigns registers on chosen segments without control flow, referred to as traces [Eis15]. This allows for fast algorithm and can use different allocation strategies based on the trace properties. However, choosing good traces is non-trivial without runtime profiling information, and the resulting code quality is highly dependent on the trace selection [Eis15].

As TPDE [SKE26] does not use have access to runtime profiling information, this approach is not suitable for our use case. Our approach allocates across control flow, without runtime information under the assumption that loops will be executed multiple times.

4.4 SSA Register Allocation

By utilizing the favorable properties of SSA form, these strategies can avoid the runtime costs of traditional graph coloring approaches. Although adapting these results for real architectures with constrained instructions and no swap instructions requires live range splitting and additional modifications to the IR [Hac07]. Such modifications would be difficult to integrate into TPDE, which cannot modify the underlying IR, in addition TPDE lacks the information about constraints before codegeneration.

Instead of the previously necessary modifications to the program, Colombet et al. [Col+11] introduce the notion of repairing if an irregularity is encountered. By allowing the register set to deviate at such points, they present a simpler algorithm that falls back to graph coloring in situations where a repairing using parallel moves along is not possible.

We adopt a similar approach to Colombet et al. [Col+11] by introducing preferred registers and repairing around constraints, however we do not use repairing across all operands of an instruction, thereby limiting our code quality for highly constrained instructions. For calls, our approach is equivalent, although we cannot fall back to a graph coloring approach should the repairing fail.

5 Evaluation

5.1 Correctness

We verify the correctness of our implementation by using the LLVM test suite and by running the SPEC CPU2017 integer benchmarks. Except for some LLVM test cases that contain unsupported instructions, all tests pass successfully on AArch64 and x86-64. Which we assume is a good proxy for the correctness of our implementation.

5.2 Setup

We evaluate the performance of our implementation by measuring the back-end compile-time and the run-time performance of LLVM-IR generated by LLVM using the optimized (`-O1`) frontend pipeline. Our implementation[1] is compared against the original version of TPDE[2] and against both the LLVM-O0 and LLVM-O1 back-end, the former two focusing on compile-time and the latter being an optimizing back-end. As benchmark programs, we use the SPEC CPU2017 integer benchmarks on x86-64 and AArch64. Our x86-64 machine is an AMD Ryzen 5 5600X with 32 GiB of RAM running Linux 6.18.12; Our AArch64 machine is an Apple M1 with 16 GiB of memory, using only the four performance cores, running Asahi Linux 6.17.12. We use Clang/LLVM version 20.1.8.

All runtime performance measurements are performed twice, with the average time being reported. Compile times are calculated from 5 runs, discarding the slowest and fastest runs. We then report the average of the remaining three runs.

As both architectures show similar results, we will refer to the `x86-64` version, unless otherwise noted.

5.3 Runtime Performance

First, we compare our solution to the LLVM back-ends, as well the original TPDE on optimized IR. The produced IR uses ϕ nodes and is in SSA form with relatively long

[1] Commit ff5f13a5
[2] Commit 930ff2b

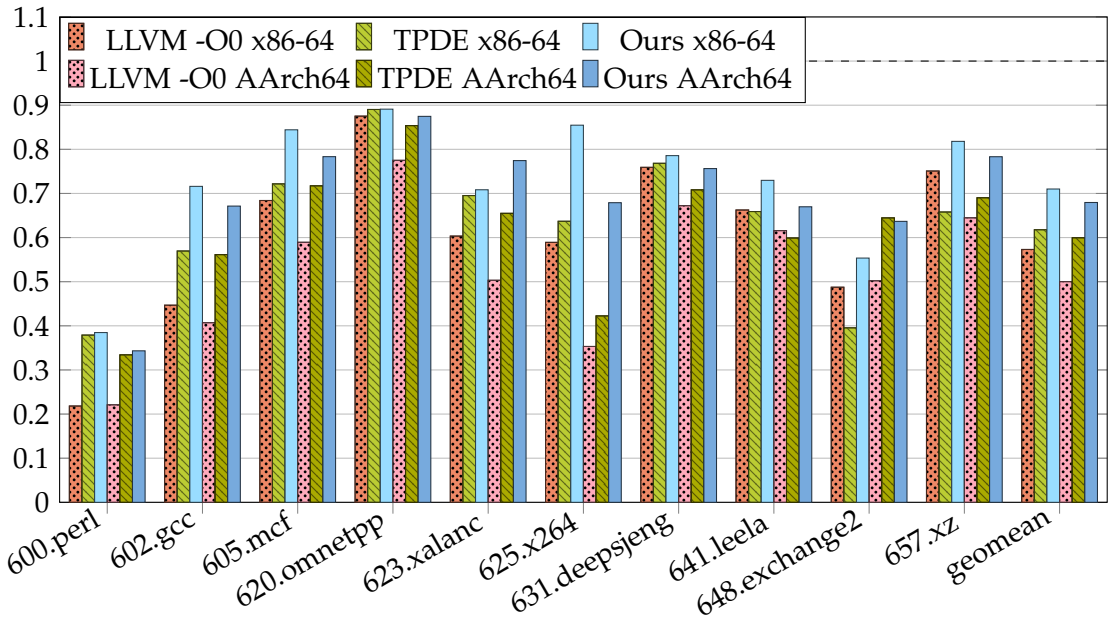


Figure 5.1: SPEC CPU2017 integer runtime speedup relative to clang-O1 backend with optimized IR.

live ranges, which makes it a good candidate for comparison.

Figure 5.1 shows an overall improvement in the runtime performance of 24% (0.573× vs. 0.710×) for our work compared to llvm-O0 backend, and a improvement of 15% (0.618× vs. 0.710×) for our work compared to the original TPDE. Compared to llvm-O1 backend, which has a better instruction selection, register allocation and performs additional optimization passes on the IR, our work is 30% slower.

In particular relative performances vary across benchmarks, with some showing no significant improvement when compared to TPDE, notably `600.perl` and mehr.

We will use `600.perl` as an example for some problems resulting from using the optimized IR as a basis for register allocation.

Suboptimal structures resulting from LLVM-O1 pipeline

The clang and LLVM-O1 frontend pipelines perform multiple transformations, but the resulting IR was not intended to be used directly without further transformations. For example the loop invariant code motion pass (LICM) moves instructions outside of loops whenever possible. This results in many pointer offsets (`getelementptr`) to be moved to the beginning of functions. Additionally, there is no deduplication pass for

```
early_block:
%strbeg2306 = getelementptr i8, ptr %reginfo, i64 8
%strbeg1235 = getelementptr i8, ptr %reginfo, i64 8
%strend = getelementptr i8, ptr %reginfo, i64 16
%not = icmp eq ptr %cur_eval.1.ph, null
                                          ⋮
inner_blocks:
%strbegval = load ptr, ptr %strbeg1235
%strendval = load ptr, ptr %strend
                                          ⋮
br i1 %not, ...
```

Listing 5.1: Excerpt from the function `S_regmatch` from perlbench IR (simplified for readability). Showing structures resulting from loop invariant code motion.

values after LICM, which results in many duplicate values for the same pointer offset. For the function `S_regmatch` from `600.perl`, which is its bottleneck, we see at least 7000 usages of such values.

This is highly problematic for our implementation as each of these values is compiled at an early block of the function and then spilled as they are not used for most of the program. Then on every load the address need to be loaded from the stack (where the same value is duplicated in many slots) and only then can the actual value be loaded. On AArch64 the amount of such values also results in an additional instruction for calculating the stack slot address, as the offset is too large to be encoded as an immediate value in the instruction, due to the thousands of values on the stack.

A better solution would be to keep the base point in a register, as it is used often throughout the program and then compute the offsets on demand, which are typically small enough to be encoded as immediate values in the instruction. This is the approach taken by the LLVM-O1 back-end, which results in much better performance.

Similarly, some comparisons are moved far from the branches that actually use them, which means we are no longer able to fuse the comparison with the branch instruction, which results in more spills and more unnecessary instructions.

As a result most of the generated code by both TPDE and our implementation is spills, reloads and connecting code. Fixing this for pointer offsets may be possible by internally tracking the base pointer but would require significant changes to the implementation. For other values this is not realistically resolvable without reimplementing an optimization pipeline, which removes the main advantage of TPDE, namely its low number of passes.

Another pattern is illustrated in Listing 5.2, where we see many small basic blocks,

```
cond.false409:
%cmp410 = icmp ult i64 %x, 67108864,
br i1 %cmp410, label %cond.end431, label %cond.false413
cond.false413:
%cmp414 = icmp ult i64 %x, 2147483648,
br i1 %cmp414, label %cond.end431, label %cond.false417
                                              ⋮
cond.end431:
%y = phi i64 [ 0, %cond.false409 ], [ 1, %cond.false413 ], ...
```

Listing 5.2: Excerpt from the function `S_regmatch` from perlbench. Showing very small basic blocks, chained together to utilize phi nodes.

each containing a single comparison and a branch. LLVM-O1 combines these small blocks into a single label with multiple jumps. Our implementation generates all of them separately and also generate connecting code to the fending block every time resulting in many duplicated spills, reloads and moves.

Such patterns are found on the hot functions of 600.*perl*, 620.*omnetpp* and 623.*xalanc* doublecheck, which are also the benchmarks showing little performance improvement for our version.

Other Limitations

In addition to the aforementioned issues, there are some other limitations of our implementation that may be contributing to the performance gap to llvm-O1. Since we lack a coalescing analysis, we generally need separate registers for the phi and its next-incoming (see *rax*,*r*9 in the left snippet of Figure 5.2), which results in a higher register pressure and leads to worse instruction selection on `x86` since we can't use the add and sub instruction as they overwrite their source operands. Additionally, our implementation currently splits all conditional branches, leading to more jumps and a worse code layout.

Since we spill all non-loop variables when entering a loop header, this can result in unnecessary spills if the loop has a low register pressure (see Figure 5.2). Since we don't know the actual register pressure at the spill stage it is difficult to design a better heuristic for this issue, as an additional spill/reload in the loop will be more expensive than the unneeded spills outside the loop.

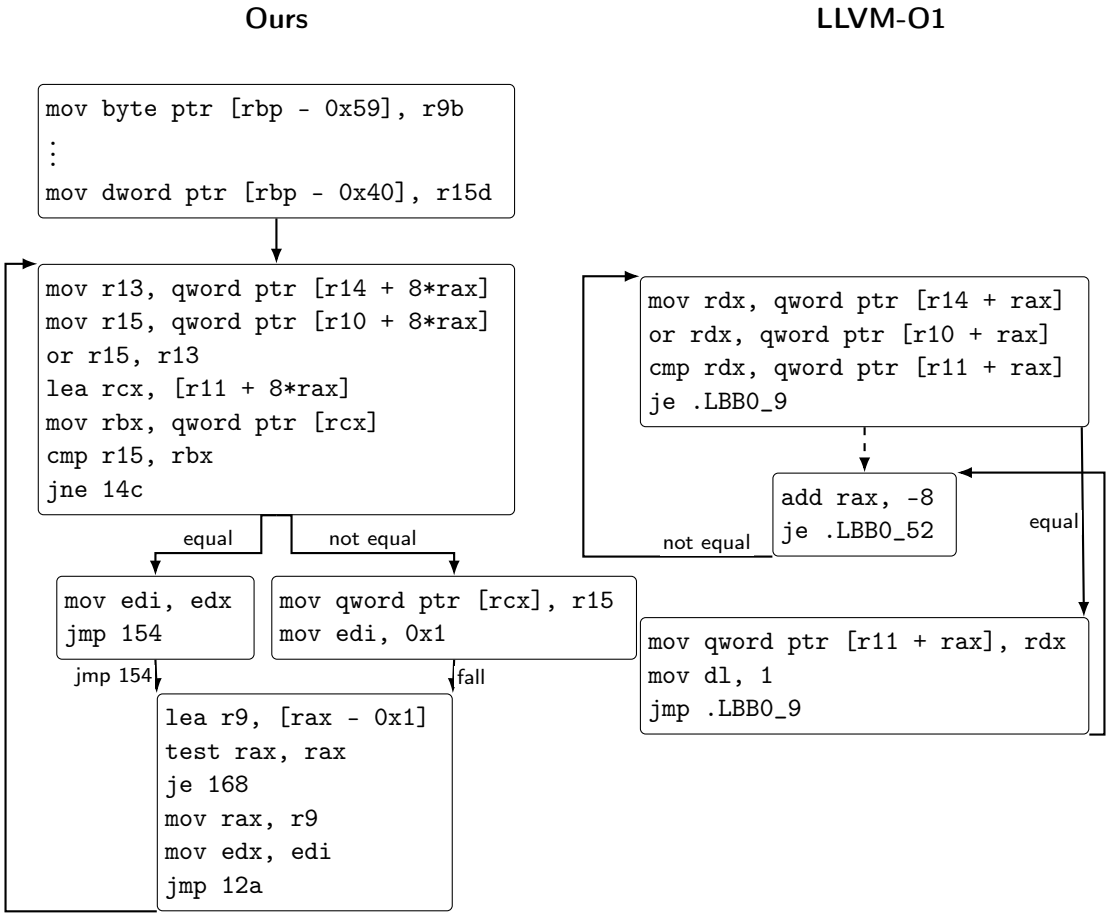


Figure 5.2: Example of a small loop from `bitmap_elt_ior`, one of the critical functions of `602.gcc`. Generated by our implementation and llvm-O1. We can clearly see a better instruction selection and multiple more optimizations such as the loop counter. Our implementation results in more instructions, a higher register pressure in the loop and adds spills before the loop entry.

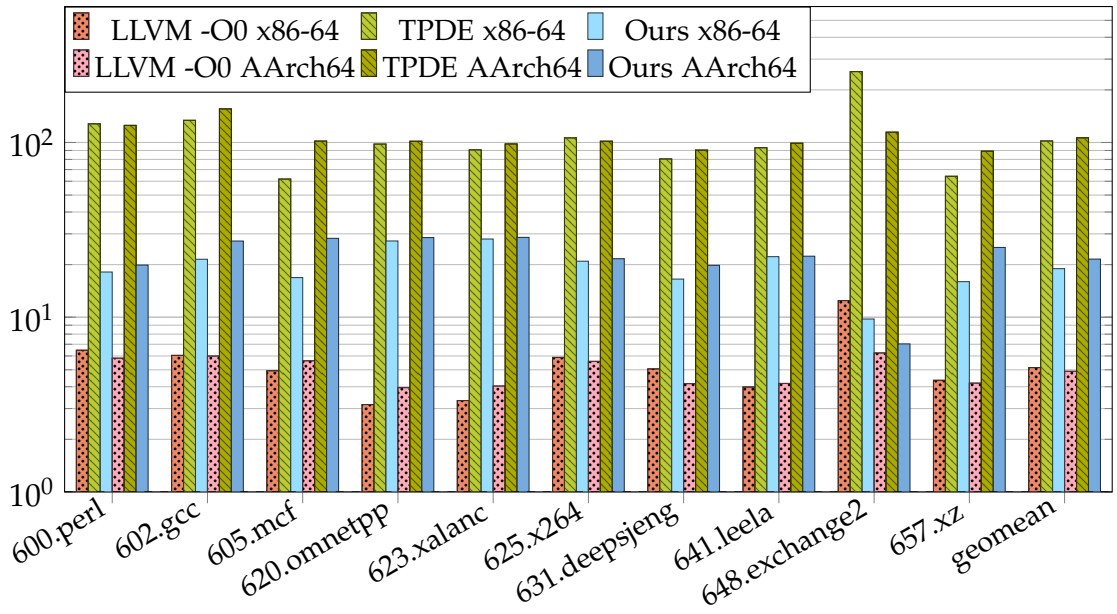


Figure 5.3: SPEC CPU2017 integer compile time speedup relative to clang-O1 backend with optimized IR. (log scale)

5.4 Compile time

We see an overall compile-time penalty of around 5.4 times (102.0× vs 19.00×) compared to TPDE, while still remaining 3.7 times faster than llvm-O0 (19.00× vs 5.150×).

The additional 2 passes through the IR, as well as the register set matching between basic blocks are the primary sources of the compile-time increase. In particular all three sections rely on sets, which using the current implementation result in many allocations, roughly taking 20% of the compile time for `600.perl` on `x86-64` when measured using the sampling-based profiler `perf` on the combined IR from `llvm-link`, suggesting an implementation issue.

5.5 Code size

When comparing sizes of the `.text` segment of the generated binaries, we see an increase of 12% for our implementation compared to TPDE (geomean 2.05× vs 1.83×), both being around twice the size of llvm-O1. Usually we expect a smaller code size for better register allocation, but in this case the splitting of jumps as well as the matching of register sets for all branches leads to an increase in code size. On AArch64 we

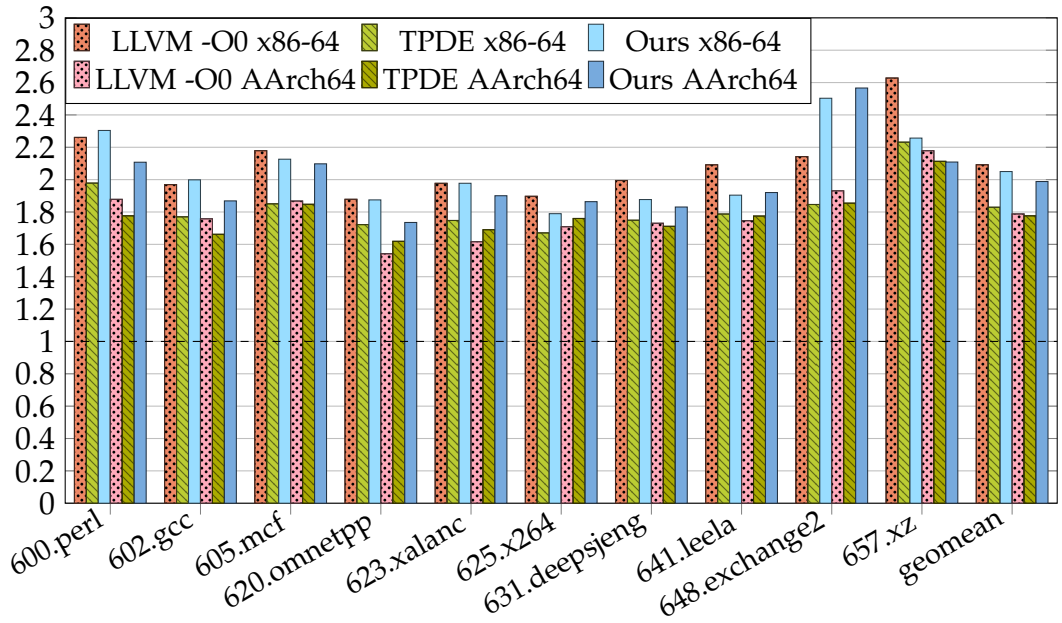


Figure 5.4: SPEC CPU2017 integer text size relative to clang-O1 backend with optimized IR.

see a worse performance when compared to LLVM-O0, which is due to the usage of `GlobalISel` for the unoptimized pipeline as opposed to `FastISel`.

The primary way to avoid this code size increase would be to optimize scenarios similar to Listing 5.2, by generating most of the connecting code at the beginning of the merge block instead of at each of its predecessors.

Matching the code size of llvm-O1 is not possible due to the superior instruction selection and additional optimizations performed by llvm-O1 (see Figure 5.2).

5.6 Discussion

Our results show, that we were able to achieve a moderate improvement in runtime performance compared to the original TPDE, while still remaining significantly faster than llvm-O0. However, there is still a significant gap to llvm-O1, which is difficult to close without more elaborate analyses. Especially in the worst performing benchmarks, where we see no improvement while incurring a large compile-time penalty. Some compile-time gap to TPDE may be closed by optimizing the implementation, specifically around minimizing the number of allocations.

```
%i = alloca i32, align 4
%j = alloca i32, align 4
                                                  ⋮
%58 = load i32, ptr %j, align 4
%59 = load i32, ptr %k, align 4
%cmp = icmp slt i32 %58, %59
br i1 %cmp, label %for.body, label %for.end
```

Listing 5.3: Example of IR generated by LLVM-O0, using stack slots for loop variables, that would be ϕ nodes otherwise.

Nevertheless, our implementation is a new pareto-optimum for compiling optimized IR, as it is significantly faster than llvm-O1 while still providing a runtime improvement compared to both TPDE and llvm-O0. Additionally, our implementation is independent of the IR, so it may be used as a back-end for other IRs, where problems such as the LICM may be avoided.

5.7 Unoptimized IR

When running the same benchmarks using unoptimized (-O0) IR, we see no runtime performance improvement between TPDE and our work, while still incurring a similar compile time penalty of 4 times (geomean 4.90× vs. 19.34×). As our analyses were designed for IR with long live ranges and phi nodes, applying them to unoptimized IR.

The IR generated by LLVM-O0, uses stack slots using `alloca` instead of ϕ nodes and loads and store to them. Therefore, most values have a short life range and functions rarely have a high register pressure, which prevents our approach from improving the code generation. Additionally, values with exactly one use represent the worst case for our liveness analysis, as they still generate an allocation, which doesn't get amortized by multiple uses.

6 Conclusion and Future Work

In this thesis we presented a novel register allocation approach built on top of TPDE, allowing for better code generation, while not requiring information on the specific instruction semantics, and keeping the core single-pass code generation. We introduced a lifetime and next-use analysis, as well as a spill analysis that approximates register pressure and optimizes spill locations around loops. During code generation, we introduced a repairing mechanism to preserve values across constrained instructions, by inserting moves.

On SPECint 2017 we achieve a. 24% runtime improvement over the LLVM -O0 backend, while keeping a three times faster compile time on optimized LLVM IR. Although this also incurs a 5 times compile time increase compared to TPDE. Despite the overall improvements, we see no difference for some benchmarks, due to the limitations of our approach when using optimized LLVM-IR, which was not designed to be used for code generation without further transformations.

Future work may address the limitations of our approach, by implementing additional analyses, specifically around `getelementptr` instructions with constant offsets, which are a limiting factor for our approach. Storing such values as offsets from a base pointer would enable better code generation especially in the worst performing benchmarks, but would potentially require additional modification to the base TPDE framework, which is intended to be IR agnostic. Additionally, the code quality may be improved by implementing a more elaborate repairing mechanism, that can handle shuffling multiple operands for a constrained instruction. Finally, a better heuristic may avoid spills for loops with low register pressure, as seen in Figure 5.2.

The compile time overhead may be reduced by optimizing our implementation, in particular by minimizing the number of allocations caused by set operations.

Abbreviations

SSA Static Single Assignment
RA Register Allocation
GC Graph coloring
IR Intermediate representation
LICM Loop invariant code motion
CFG Control flow graph

List of Figures

List of Tables

Bibliography

[BH09] M. Braun and S. Hack. "Register Spilling and Live-Range Splitting for SSA-Form Programs." In: *Compiler Construction*. Ed. by O. de Moor and M. I. Schwartzbach. Berlin, Heidelberg: Springer Berlin Heidelberg, 2009, pp. 174–189. ISBN: 978-3-642-00722-4.

[Boi+11] B. Boissinot, F. Brandner, A. Darte, B. D. de Dinechin, and F. Rastello. "A Non-iterative Data-Flow Algorithm for Computing Liveness Sets in Strict SSA Programs." In: *Programming Languages and Systems*. Ed. by H. Yang. Berlin, Heidelberg: Springer Berlin Heidelberg, 2011, pp. 137–154. ISBN: 978-3-642-25318-8.

[CD06] K. Cooper and A. Dasgupta. "Tailoring graph-coloring register allocation for runtime compilation." In: *International Symposium on Code Generation and Optimization (CGO'06)*. 2006, 11 pp.–49. DOI: 10.1109/CGO.2006.35.

[Cha+81] G. J. Chaitin, M. A. Auslander, A. K. Chandra, J. Cocke, M. E. Hopkins, and P. W. Markstein. "Register allocation via coloring." In: *Computer Languages* 6.1 (1981), pp. 47–57. ISSN: 0096-0551. DOI: https://doi.org/10.1016/0096-0551(81)90048-5.

[Col+11] Q. Colombet, B. Boissinot, P. Brisk, S. Hack, and F. Rastello. "Graph-coloring and treescan register allocation using repairing." In: *Proceedings of the 14th International Conference on Compilers, Architectures and Synthesis for Embedded Systems*. CASES '11. Taipei, Taiwan: Association for Computing Machinery, 2011, pp. 45–54. ISBN: 9781450307130. DOI: 10.1145/2038698.2038708.

[Eis15] J. Eisl. "Trace register allocation." In: *Companion Proceedings of the 2015 ACM SIGPLAN International Conference on Systems, Programming, Languages and Applications: Software for Humanity*. SPLASH Companion 2015. Pittsburgh, PA, USA: Association for Computing Machinery, 2015, pp. 21–23. ISBN: 9781450337229. DOI: 10.1145/2814189.2814199.

[FW02] C. Fu and K. Wilken. "A faster optimal register allocator." In: *35th Annual IEEE/ACM International Symposium on Microarchitecture, 2002. (MICRO-35). Proceedings.* 2002, pp. 245–256. DOI: 10.1109/MICRO.2002.1176254.

[GJ90] M. R. Garey and D. S. Johnson. *Computers and Intractability; A Guide to the Theory of NP-Completeness*. USA: W. H. Freeman & Co., 1990. ISBN: 0716710455.

[Hac07] S. Hack. "Register allocation for programs in SSA Form." PhD thesis. 2007. 123 pp. ISBN: 978-3-86644-180-4. DOI: 10.5445/KSP/1000007166.

[HGG06] S. Hack, D. Grund, and G. Goos. "Register Allocation for Programs in SSA-Form." In: *Compiler Construction*. Ed. by A. Mycroft and A. Zeller. Berlin, Heidelberg: Springer Berlin Heidelberg, 2006, pp. 247–262. ISBN: 978-3-540-33051-6.

[LA04] C. Lattner and V. Adve. "LLVM: A Compilation Framework for Lifelong Program Analysis and Transformation." In: San Jose, CA, USA, Mar. 2004, pp. 75–88.

[Neu11] T. Neumann. "Efficiently compiling efficient query plans for modern hardware." In: *Proc. VLDB Endow.* 4.9 (June 2011), pp. 539–550. ISSN: 2150-8097. DOI: 10.14778/2002938.2002940.

[PEK97] M. Poletto, D. R. Engler, and M. F. Kaashoek. "tcc: a system for fast, flexible, and high-level dynamic code generation." In: *Proceedings of the ACM SIGPLAN 1997 Conference on Programming Language Design and Implementation*. PLDI '97. Las Vegas, Nevada, USA: Association for Computing Machinery, 1997, pp. 109–121. ISBN: 0897919076. DOI: 10.1145/258915.258926.

[PM04] J. Park and S.-M. Moon. "Optimistic register coalescing." In: *ACM Trans. Program. Lang. Syst.* 26.4 (July 2004), pp. 735–765. ISSN: 0164-0925. DOI: 10.1145/1011508.1011512.

[PVC01] M. Paleczny, C. Vick, and C. Click. "The java hotspotTM server compiler." In: *Proceedings of the 2001 Symposium on JavaTM Virtual Machine Research and Technology Symposium - Volume 1*. JVM'01. Monterey, California: USENIX Association, 2001, p. 1.

[RSL08] L. Rideau, B. P. Serpette, and X. Leroy. "Tilting at Windmills with Coq: Formal Verification of a Compilation Algorithm for Parallel Moves." In: *J. Autom. Reason.* 40.4 (May 2008), pp. 307–326. ISSN: 0168-7433. DOI: 10.1007/s10817-007-9096-8.

[SKE26] T. Schwarz, T. Kamm, and A. Engelke. "TPDE: A Fast Adaptable Compiler Back-End Framework." In: *2026 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. 2026, pp. 427–439. DOI: 10.1109/CGO68049.2026.11395208.

[WF10] C. Wimmer and M. Franz. "Linear scan register allocation on SSA form." In: *Proceedings of the 8th Annual IEEE/ACM International Symposium on Code Generation and Optimization*. CGO '10. Toronto, Ontario, Canada: Association for Computing Machinery, 2010, pp. 170–179. ISBN: 9781605586359. DOI: 10.1145/1772954.1772979.

[WM05] C. Wimmer and H. Mössenböck. "Optimized interval splitting in a linear scan register allocator." In: *Proceedings of the 1st ACM/USENIX International Conference on Virtual Execution Environments*. VEE '05. Chicago, IL, USA: Association for Computing Machinery, 2005, pp. 132–141. ISBN: 1595930477. DOI: 10.1145/1064979.1064998.